

# From duplicate proofs to a reusable lemma

A small, reviewable mathlib contribution pattern

PATTERN

EXTRACT

REUSE

## 1. Spot repetition

Same order argument in ENat and ENNReal

## 2. Extract the invariant

A cofinal diagonal controls a double supremum

## 3. Replace callers

Domain proofs become rewrite + exact

## The duplicated shape

### ENat.iSup\_add\_iSup

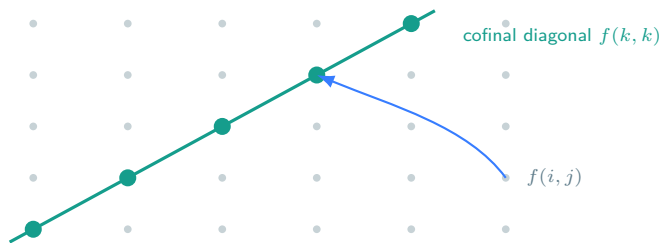
```
refine le_antisymm ?_ ?_  
refine iSup_add_iSup_le fun i j => ?_  
rcases h i j with <k, hk>  
exact le_iSup_of_le k hk
```

### ENNReal.iSup\_add\_iSup

```
refine le_antisymm ?_ ?_  
refine iSup_add_iSup_le fun i j => ?_  
rcases h i j with <k, hk>  
exact le_iSup_of_le k hk
```

## The weakest common statement

```
@[to_dual iInf2_eq_diagonal]  
theorem iSup2_eq_diagonal  
  {alpha : Type*} {iota : Sort*} [CompleteSemilatticeSup alpha]  
  (f : iota -> iota -> alpha)  
  (h : forall i j, exists k, f i j <= f k k) :  
  (iSup i, iSup j, f i j) = iSup k, f k k := by  
  ...
```



Every grid point lies below  
some point on the diagonal.

**Therefore both suprema agree.**

# Proof map in Lean 4

Two inequalities, then immediate reuse

MACHINE-CHECKED

## Goal

$$\left( \sup_i \sup_j f(i, j) \right) = \sup_k f(k, k) \quad \text{under} \quad \forall i, j, \exists k, f(i, j) \leq f(k, k).$$

Split equality

apply le\_antisymm

$\leq$  Expand both suprema Introduce arbitrary  $i, j$ .

apply sSup\_le; rintro ...

Use cofinality Choose  $k$  with  $f(i, j) \leq f(k, k)$ .

obtain <k, hk> := h i j

Insert the diagonal point  $f(i, j) \leq f(k, k) \leq \sup_k f(k, k)$ .

hk.trans (le\_sSup ...)

$\geq$  Diagonal inclusion Each  $f(k, k)$  already occurs in the double supremum. le\_sSup ...  
|>.trans (le\_sSup ...)

## Caller reduction

### Before

```
cases isEmpty_or_nonempty iota
. simp only [iSup_of_empty, ...]
. refine le_antisymm ?_ ?_
  refine iSup_add_iSup_le fun i j => ?_
  rcases h i j with <k, hk>
  exact le_iSup_of_le k hk
```

### After

```
cases isEmpty_or_nonempty iota
. simp
. simp_rw [ENat.iSup_add, ENat.add_iSup]
  exact iSup2_eq_diagonal
    (fun i j => f i + g j) h
```

## Why this is upstream-friendly

Weak assumption  
CompleteSemilatticeSup

Predictable name  
iSup2\_eq\_iSup\_diagonal

Visible reuse  
ENat + ENNReal

Dual for free  
iInf2\_eq\_iInf\_diagonal

## Review checklist

generic

discoverable

2 callers

build + lint

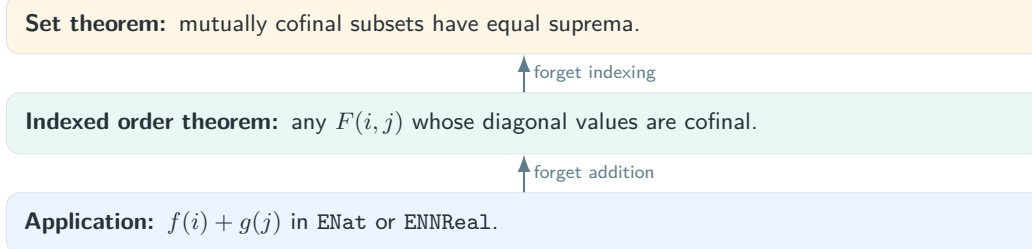
# Big idea: duplicate proofs are failed factorisations

The repeated code is evidence that one mathematical morphism has not yet been named

STRUCTURE



## The abstraction ladder



### The design rule: abstract exactly as far as the reuse evidence supports.

A good lemma forgets accidental data (addition, number type, local names) while retaining the minimal structure needed by the proof (here, a complete supremum semilattice and cofinality). Going farther can be mathematically elegant, but without callers it increases API and review cost.

## Cool fact: a two-dimensional supremum can be one-dimensional

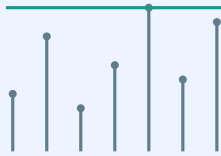
The whole grid may look larger than its diagonal, yet cofinality makes the diagonal a *lossless compression*: it sees every value up to domination. This is not a cardinality claim; it is a claim about what the supremum can detect.

# Three analogies for cofinality

Different pictures of the same invariant: maxima care about reach, not enumeration

INTUITION

## 1. Skyline sensors



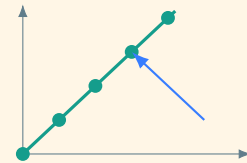
A selected sensor network suffices if every omitted reading is no higher than some selected reading. The skyline maximum is unchanged.

## 2. Express stops



An express route can ignore local stops while still reaching arbitrarily far along the line. A tail property observed on the express route controls the full route.

## 3. Synchronising clocks



Two independent indices  $(i, j)$  can be advanced to one common time  $k$ . Directedness supplies the synchronisation point.

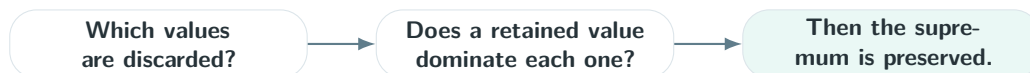
### What all three analogies preserve

Not the individual points, their order, or their number. They preserve one universal question: *is every discarded value bounded by a retained value?* If yes, taking a supremum cannot tell the full family from the retained cofinal subfamily.

### Important non-analogy

“Diagonal” here does **not** mean Cantor’s diagonal argument. No contradiction or self-reference is involved. It is the ordinary map  $k \mapsto (k, k)$ , used to replace two indices by one common upper stage.

## A reusable mental test



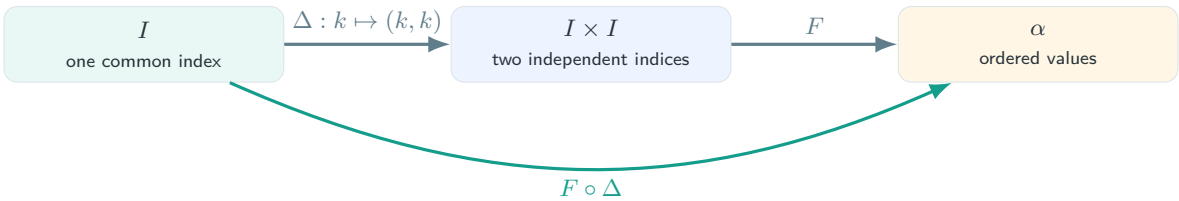
# The category-theory pattern

The order lemma is a thin-category shadow of cofinality and colimit invariance

CATEGORY VIEW

### Dictionary

| Order theory       | Category theory      | In this example                            |
|--------------------|----------------------|--------------------------------------------|
| preorder           | thin category        | one arrow $x \rightarrow y$ iff $x \leq y$ |
| least upper bound  | colimit of a diagram | $\sup_i f(i)$                              |
| cofinal subset/map | final functor        | keep only enough indices                   |
| order dual         | opposite category    | supremum $\leftrightarrow$ infimum         |



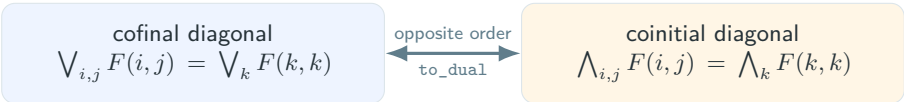
### Finality in the familiar directed case

If  $I$  is directed, every pair  $(i, j)$  admits a common upper bound  $k$ . Thus the diagonal  $\Delta : I \rightarrow I \times I$  reaches beyond every pair. For a monotone  $F$ , this gives  $F(i, j) \leq F(k, k)$ , and restricting the colimit/supremum along  $\Delta$  changes nothing.

### Structural subtlety

The Lean lemma assumes only  $\forall i, j, \exists k, F(i, j) \leq F(k, k)$ . This is *value-level cofinality*. It can hold even when the index type has no useful order at all. So the theorem is weaker and more reusable than a categorical statement asserting that  $\Delta$  itself is final as a functor between ordered index categories.

### Why the dual appears automatically



# Structural insights for API design

What this tiny lemma teaches beyond its three callers

DESIGN

## 1. Universal properties compress proofs

A supremum is characterised by upper bounds. Once the proof is phrased through that interface, `ENat`, `ENNReal`, and addition disappear.

## 2. Cofinality is semantic dead-code elimination

Non-cofinal points may matter individually, but they contribute no new information to the final supremum.

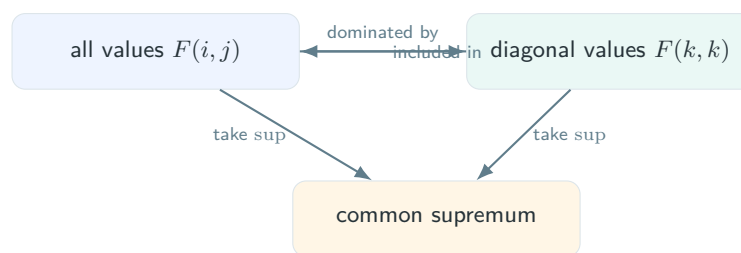
## 3. Weak assumptions widen reuse

Use `CompleteSemilatticeSup`, not a lattice, order, topology, or algebraic structure that the proof never inspects.

## 4. Duality is an API completeness test

A natural supremum theorem suggests an infimum twin. Generating it systematically aligns names and prevents a second duplicate proof.

## The commutative proof picture



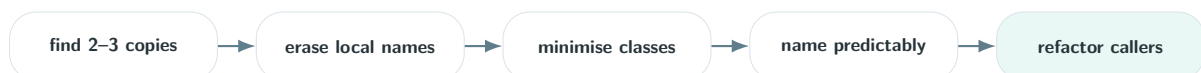
## Two inequalities, two meanings

**Inclusion** says the diagonal contributes no foreign values, so its supremum is no larger.

**Cofinality** says the omitted grid contributes no new height, so the full supremum is no larger.

Antisymmetry turns these complementary facts into equality.

## A practical extraction checklist



The strongest contribution signal is not that a lemma can be proved, but that existing code becomes shorter and conceptually more uniform after it is named.